

Automata

Időlimit: 2 másodperc

Memórialimit: 1024 MiB

Van egy N részből álló terület, amelynek helyei balról jobbra 0-tól $N - 1$ -ig vannak számozva. Minden hely magassága eltérő: az i -edik hely magassága p_i , és a $p_0, p_1, \dots, p_{N-1}$ sorozat a $0, 1, \dots, N - 1$ számok permutációja.

Az i és j **indexű** helyeket akkor nevezünk *közeli* helyeknek, ha $|i - j| \leq 1$. Minden hely közeli önmagához.

A terület egyik helyén áll egy robot. Ez a robot kétféle parancsot fogad el, amelyek a következők:

- **MAX**: a robot aktuális helyéhez közeli összes hely közül a robot következő pozíciója az az egyértelműen meghatározott hely lesz, amelynek a **legnagyobb** a magassága;
- **MIN**: a robot aktuális helyéhez közeli összes hely közül a robot következő pozíciója az az egyértelműen meghatározott hely lesz, amelynek a **legkisebb** a magassága;

Mivel minden hely közel van önmagához, ezért előfordulhat, hogy egy parancs hatására a robot ugyanazon a helyen marad.

A *program* egy véges parancssorozat, amelyben minden parancs vagy **MAX**, vagy **MIN**. Ha a robot az X helyről indul, és az S programot követi, akkor egy egyértelműen meghatározott helyen áll meg, amelyet $\text{result}(S, X)$ -szel jelölünk.

Kapsz Q kérdést, amelyek mindegyike megad egy K_i elemű lehetséges kezdőhelyből álló halmazt ($0 \leq i < Q$). Pontosabban az i -edik kérdés az $x_0, x_1, \dots, x_{K_i-1}$ helyekre vonatkozik; a robotot ezek egyikébe helyezik el, de előre nem tudni, hogy melyikbe. Minden kérdésnél meg kell állapítanod, hogy létezik-e olyan program, amely a robotot ugyanarra a végső helyre mozgatja, függetlenül attól, hogy melyik x halmazbeli helyről indul.

Formálisan: meg kell állapítanod, hogy létezik-e olyan S program, amelyre teljesül:

$$\text{result}(S, x_0) = \text{result}(S, x_1) = \dots = \text{result}(S, x_{K_i-1}).$$

A p permutáció minden kérdés esetén **ugyanaz**.

Fontos, hogy nem kell ilyen programot készítened – csak azt kell meghatároznod, hogy létezik-e ilyen.

Megvalósítás

Az első függvényt, amelyet meg kell valósítanod:

```
void initialize(std::vector<int> p)
```

- p : a 0-tól $N - 1$ -ig terjedő számok permutációja.

A második függvény, amelyet implementálnod kell:

```
bool exists_program(std::vector<int> x)
```

- x : egy tömb, amely szigorúan növekvő sorrendben adja meg a helyeket egy kérdéshez.

Az `initialize` függvényt pontosan egyszer hívják meg, még mielőtt az `exists_program` függvényt bármilyen módon meghívják.

Az `exists_program` függvényt Q -szor hívják meg, minden kérdésnél egyszer. Akkor kell `true` értéket visszaadnia, ha létezik olyan program, amelynek végrehajtása után a robot ugyanazon a helyen áll meg, függetlenül attól, hogy a megadott helyek közül melyikről indult, egyébként pedig `false` értéket kell visszaadnia.

Korlátok

- $3 \leq N \leq 2 \cdot 10^5$
- $1 \leq Q \leq 5 \cdot 10^5$
- $p_0, p_1, \dots, p_{N-1}$ a $0, 1, \dots, N - 1$ számok permutációja
- $2 \leq K_i \leq N$
- Minden kérdésre igaz, hogy $0 \leq x_0 < x_1 < \dots < x_{K_i-1} < N$
- Az összes Q kérdésre vonatkozóan a K_i értékek összege nem haladja meg a 10^6 -t.

Példák

Példa bemenet 1	Példa kimenet 1
3 3 0 2 1 2 0 2 3 0 1 2 2 1 2	111

Példa bemenet 2	Példa kimenet 2
7 3 0 4 2 1 3 5 6 3 0 1 3 3 1 3 4 2 3 6	001

Magyarázat az 1. példához

Ebben a példában $p = [0, 2, 1]$. A második kérdésnél a robot a 0., az 1. vagy a 2. helyről indulhat. Tekintsük a `[MAX]` programot.

- Ha a robot a 0. helyről indul: a 0-hoz közeli helyek a 0 és 1 indexűek. Amikor a robot végrehajtja a `MAX` parancsot, mivel $p_0 < p_1$, ezért az 1-es helyre mozog.
- Ha a robot az 1. helyről indul: az 1-hez közeli helyek a 0, 1 és 2 indexűek. Amikor a robot végrehajtja a `MAX` parancsot, mivel $p_0 < p_1$ és $p_1 > p_2$, ezért az 1-es helyen marad.
- Ha a robot a 2. helyről indul: a 2-höz közeli helyek az 1 és a 2 indexűek. Amikor a robot végrehajtja a `MAX` parancsot, mivel $p_1 > p_2$, ezért a 1-es helyre lép.

Ezért létezik olyan program, amelynek végrehajtása után a robot mindhárom kiindulási cellából az 1. helyen áll meg, így a kérdésre adott válasz `true`. Más programok is működnek, például a `[MIN, MAX]`, a `[MIN, MIN, MAX, MIN, MAX, MIN]` és a `[MAX, MIN, MAX]`.

Magyarázat a 2. példához

Ebben a példában $p = [0, 4, 2, 1, 3, 5, 6]$.

Az első két kérdés esetében bizonyítható, hogy nincs olyan program, amelynek segítségével a robot minden adott kiindulási helyről ugyanarra a helyre érkezne, így mindkettőre `false` a válasz.

Nézzük az utolsó kérdést, amelynél a robot a 3-as vagy a 6-os helyről indulhat, és tekintsük a `[MAX, MAX, MAX]` programot.

- Ha a robot a 3. helyről indul: a 3-hoz közeli helyek a 2, 3 és 4 indexűek. Mivel $p_3 < p_4$ és $p_2 < p_4$, ezért a robot a 4-es helyre mozog. A következő két parancsot ugyanígy végrehajtva a robot a 6-os helyen áll meg.
- Ha a robot a 6. helyről indul, akkor végig a 6-os helyen marad, és ott is áll meg.

Ezért erre a kérdésre a válasz `true`. A `[MIN, MAX, MAX, MAX]` program is működik. Más programok azonban nem biztos, hogy működnek. Például, ha a program `[MAX, MAX]`, akkor a robot a 3-as celláról indulva az 5-ös helyen áll meg, de a 6-os celláról indulva a 6-os helyen áll meg.

Részfeladatok

Részfeladat	Pontszám	N	Q	K_i	További korlátok
0	0	–	–	–	A példák.
1	3	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$= 2$	Ha egy kérdésre adott válasz igen, akkor létezik olyan érvényes program, amely kizárólag 1 parancsot használ.
2	7				Ha egy kérdésre adott válasz igen, akkor létezik olyan érvényes program, amely legfeljebb 5 parancsot használ.
3	9	≤ 100	≤ 500	$= 2$	–
4	17	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$= 2$	–
5	7	$\leq 2 \cdot 10^5$			$x_1 = x_0 + 1$ minden kérdésre.
6	8	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$\leq N$	–
7	13	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$\leq N$	A permutáció először növekvő, majd csökkenő, végül ismét növekvő sorrendű. Formálisan, léteznek olyan a, b egész számok, amelyekre $0 < a < b < N - 1$, és $p_0 < p_1 < \dots < p_a > p_{a+1} > \dots > p_b < p_{b+1} < \dots < p_{N-1}$.
8	13				$p_0 < p_1 > p_2 < p_3 > \dots < p_{N-1}$, ha N páros, és $p_0 < p_1 > p_2 < p_3 > \dots > p_{N-1}$, ha N páratlan.
9	23				–

Mintaértékelő

A bemenet formátuma a következő:

- 1. sor: két egész szám – az N és a Q értéke;
- 2. sor: N egész szám, $p_0, p_1, \dots, p_{N-1}$, ahol p_i az i -edik hely magassága;
- $3 + i$. sor: az i . kérdéshez megadott helyek – egy K_i egész szám, amelyet ugyanabban a sorban K_i egész szám követi $x_0, x_1, \dots, x_{K_i-1}$, a robot lehetséges kiindulási helyei.

A kimenet formátuma a következő:

- 1. sor: egy Q hosszúságú bináris karakterlánc, amelynek i -edik karaktere 1 , ha az i -edik kérdésre adott válasz `true`, egyébként 0 .